

✓ Premier League Transfer Value Predictor

Data Science & Machine Learning Project

Goal: Estimate a football player's transfer fee using age, position, playing time, attacking output, passing, progression, shooting, and defensive performance statistics.

This project combines:

- **Transfermarkt** transfer records for the target variable (actual transfer fee)
- **FBref** player-performance statistics for explanatory features
- **Pandas / NumPy** for data preparation
- **Matplotlib / Seaborn** for exploratory analysis
- **Scikit-learn** for preprocessing, model training, validation, tuning, and evaluation

Important: This is a data-science project, not a claim that transfer fees can be determined purely from on-field statistics. Transfer prices are also affected by factors such as contract length, club finances, market conditions, player reputation, age, and negotiation context. The model learns patterns from the available dataset.

1. Project Overview

Problem Statement

Football transfer fees can vary substantially even among players with similar playing statistics. The purpose of this project is to investigate whether a player's observable performance profile can be used to estimate his transfer fee.

Objective

Build a regression model that predicts a player's **transfer fee in euros** from football performance and player-profile features.

Main Questions

1. How should transfer and performance datasets be cleaned and combined?
2. Which player-performance variables contain useful predictive information?
3. How well can a Random Forest regression model estimate transfer fees?
4. Which features are most influential according to permutation importance?
5. How does the model perform against a simple mean-value baseline?

Final Modeling Dataset

The original workflow produced **91 matched players** after cleaning the 2025 Premier League incoming-transfer data and matching those players to FBref records. The resulting modeling table contains **33 columns**, including the player identifier, transfer fee, age, position, and performance features.

2. Data Sources

Transfermarkt

Seven transfer datasets covering **2019–2025** were loaded during the original project. The 2025 dataset is used for the final modeling task.

For the final target dataset, we retain:

- Premier League transfers
- Incoming movements
- Non-loan transfers
- Records with a known transfer fee

This reduced the 2025 Premier League incoming records to **138 non-loan transfers with known fees** before matching with FBref.

FBref

Two FBref files were inspected. The lighter **165-column dataset** was selected as the main performance source because it contains the player-level performance statistics needed for the model without the additional duplicated metadata contained in the larger file.

The original FBref file contained **2,854 rows and 165 columns**.

3. Imports

```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from difflib import get_close_matches

from sklearn.compose import ColumnTransformer
from sklearn.dummy import DummyRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.inspection import permutation_importance
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.model_selection import (
    train_test_split,
    KFold,
    cross_validate,
    GridSearchCV
)
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder

pd.set_option("display.max_columns", 100)
pd.set_option("display.max_colwidth", 40)

RANDOM_STATE = 42
```

4. Upload the Project Data

The notebook expects these files in the Colab working directory:

```
2019.csv
2020.csv
```

```
2021.csv
2022.csv
2023.csv
2024.csv
2025.csv
players_data_light-2024_2025.csv
players_data-2024_2025.csv
```

The larger FBref file is inspected for comparison but is not used in the modeling pipeline.

```
# Run this cell in Google Colab if the files are not already present.
```

```
try:
    from google.colab import files
    uploaded = files.upload()
    print("Uploaded files:", list(uploaded.keys()))
except ImportError:
    print("Google Colab upload widget is unavailable in this environment.")
    print("Make sure the required CSV files are present in the working director
```

Choose Files 9 files

```
2025.csv(text/csv) - 66068 bytes, last modified: 9/19/2026 - 100% done
2024.csv(text/csv) - 76572 bytes, last modified: 9/19/2026 - 100% done
2023.csv(text/csv) - 79490 bytes, last modified: 9/19/2026 - 100% done
2022.csv(text/csv) - 78384 bytes, last modified: 9/19/2026 - 100% done
2021.csv(text/csv) - 66515 bytes, last modified: 9/19/2026 - 100% done
2020.csv(text/csv) - 66636 bytes, last modified: 9/19/2026 - 100% done
2019.csv(text/csv) - 71574 bytes, last modified: 9/19/2026 - 100% done
players_data_light-2024_2025.csv(text/csv) - 1368966 bytes, last modified: 6/23/2025 - 100%
done
players_data-2024_2025.csv(text/csv) - 2648432 bytes, last modified: 6/23/2025 - 100% done
Saving 2025.csv to 2025.csv
Saving 2024.csv to 2024.csv
Saving 2023.csv to 2023.csv
Saving 2022.csv to 2022.csv
Saving 2021.csv to 2021.csv
Saving 2020.csv to 2020.csv
Saving 2019.csv to 2019.csv
Saving players_data_light-2024_2025.csv to players_data_light-2024_2025.csv
Saving players_data-2024_2025.csv to players_data-2024_2025.csv
Uploaded files: ['2025.csv', '2024.csv', '2023.csv', '2022.csv', '2021.csv', '20
```

✓ 5. Load the Datasets

```
transfer_2019 = pd.read_csv("2019.csv")
transfer_2020 = pd.read_csv("2020.csv")
transfer_2021 = pd.read_csv("2021.csv")
transfer_2022 = pd.read_csv("2022.csv")
transfer_2023 = pd.read_csv("2023.csv")
transfer_2024 = pd.read_csv("2024.csv")
transfer_2025 = pd.read_csv("2025.csv")

df_fbref = pd.read_csv("players_data_light-2024_2025.csv")
df_fbref2 = pd.read_csv("players_data-2024_2025.csv")

transfer_datasets = {
    2019: transfer_2019,
    2020: transfer_2020,
    2021: transfer_2021,
    2022: transfer_2022,
    2023: transfer_2023,
    2024: transfer_2024,
    2025: transfer_2025,
```

```
}

for season, df in transfer_datasets.items():
    print(f"{season}: {df.shape}")

print("\nFBref light:", df_fbref.shape)
print("FBref full :", df_fbref2.shape)
```

```
2019: (530, 16)
2020: (491, 16)
2021: (492, 16)
2022: (573, 16)
2023: (581, 16)
2024: (559, 16)
2025: (480, 16)
```

```
FBref light: (2854, 165)
FBref full : (2854, 267)
```

✓ 6. Inspect the FBref Datasets

The two FBref files contain similar player observations, but the larger file contains many additional columns and duplicated metadata from different statistical categories.

The lighter dataset is therefore used as the main modeling source.

```
print("FBref light object columns:")
print(df_fbref.select_dtypes(include="object").columns.tolist())

print("\nFBref light information:")
df_fbref.info()

print("\nFBref full information:")
df_fbref2.info()

display(df_fbref.head(3))
```

FBref light object columns:
['Player', 'Nation', 'Pos', 'Squad', 'Comp']

FBref light information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2854 entries, 0 to 2853
Columns: 165 entries, Rk to AvgDist
dtypes: float64(61), int64(99), object(5)
memory usage: 3.6+ MB

FBref full information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2854 entries, 0 to 2853
Columns: 267 entries, Rk to AvgDist
dtypes: float64(111), int64(121), object(35)
memory usage: 5.8+ MB

	Rk	Player	Nation	Pos	Squad	Comp	Age	Born	MP	Starts	Min
0	1	Max Aarons	eng ENG	DF	Bournemouth	eng Premier League	24.0	2000.0	3	1	86
1	2	Max Aarons	eng ENG	DF,MF	Valencia	es La Liga	24.0	2000.0	4	1	120
2	3	Rodrigo Abajas	es ESP	DF	Valencia	es La Liga	21.0	2003.0	1	1	65

3 rows × 165 columns

✓ 7. Inspect the 2025 Transfer Data

Before filtering, the 2025 transfer table contains both incoming and outgoing movements, loans, and records with missing fees.

We first inspect these properties so that the final filtering rules are transparent.

```
print("Movement types:")
display(transfer_2025["movement"].value_counts())

print("Missing transfer fees:", transfer_2025["fee"].isna().sum())
print("Non-missing transfer fees:", transfer_2025["fee"].notna().sum())

print("\nLoan status:")
display(transfer_2025["is_loan"].value_counts())

print("\n2025 dataset columns:")
print(transfer_2025.columns.tolist())
```

Movement types:

	count
movement	
out	317
in	163

dtype: int64

Missing transfer fees: 35

Non-missing transfer fees: 445

Loan status:

	count
is_loan	
0	308
1	172

dtype: int64

2025 dataset columns:

['season', 'league', 'club', 'window', 'movement', 'player_name', 'player_id', ' '

✓ 8. Create the Clean 2025 Premier League Transfer Dataset

The target population is defined as:

- `league == "Premier League"`
- `movement == "in"`
- `is_loan == 0`
- transfer fee is available

This creates the pool of permanent Premier League incoming transfers used for matching.

```
transfer_2025_clean = transfer_2025[
    (transfer_2025["league"] == "Premier League") &
    (transfer_2025["movement"] == "in") &
    (transfer_2025["is_loan"] == 0) &
    (transfer_2025["fee"].notna())
].copy()

print("Clean 2025 transfer records:", len(transfer_2025_clean))
print("Missing fees:", transfer_2025_clean["fee"].isna().sum())
print("Loans remaining:", transfer_2025_clean["is_loan"].sum())
print("Movement types:", transfer_2025_clean["movement"].unique())
print("Leagues:", transfer_2025_clean["league"].unique())

display(
    transfer_2025_clean[
        [
            "season", "player_name", "player_id", "club", "age",
            "position", "market_value", "dealing_club", "fee", "is_loan"
        ]
    ].head(20)
)
```

Clean 2025 transfer records: 138
Missing fees: 0
Loans remaining: 0
Movement types: ['in']
Leagues: ['Premier League']

	season	player_name	player_id	club	age	position	market_value	d
0	2025	Bafodé Diakité	463605	AFC Bournemouth	24	Centre-Back	28000000.0	
1	2025	Djordje Petrovic	465555	AFC Bournemouth	25	Goalkeeper	20000000.0	
2	2025	Ben Gannon-Doak	719673	AFC Bournemouth	19	Right Winger	14000000.0	
3	2025	Amine Adli	532776	AFC Bournemouth	25	Left Winger	18000000.0	
4	2025	Veljko Milosavljevic	989112	AFC Bournemouth	18	Centre-Back	1000000.0	
5	2025	Adrien Truffert	654539	AFC Bournemouth	23	Left-Back	15000000.0	S
25	2025	Martin Zubimendi	423440	Arsenal FC	26	Defensive Midfield	60000000.0	F
26	2025	Eberechi Eze	479999	Arsenal FC	27	Attacking Midfield	55000000.0	C
27	2025	Viktor Gyökeres	325443	Arsenal FC	27	Centre-Forward	75000000.0	
28	2025	Noni Madueke	503987	Arsenal FC	23	Right Winger	40000000.0	
29	2025	Cristhian Mosquera	646750	Arsenal FC	21	Centre-Back	30000000.0	
30	2025	Christian Nørgaard	148367	Arsenal FC	31	Defensive Midfield	11000000.0	
31	2025	Kepa Arrizabalaga	192279	Arsenal FC	30	Goalkeeper	10000000.0	

9. Match Transfermarkt Players to FBref

A direct name match initially found fewer matches than expected because names can differ across data sources.

Examples include:

- accented vs. unaccented names
- different punctuation
- minor spelling variations

The original project manually verified close matches rather than blindly accepting fuzzy matches.

```
transfer_names = set(
    transfer_2025_clean["player_name"].dropna().str.strip()
)
fbref_names_all = set(
    df_fbref["Player"].dropna().str.strip()
)

exact_matches = transfer_names & fbref_names_all
```

```

print("Transfermarkt players:", len(transfer_names))
print("All FBref players:", len(fbref_names_all))
print("Exact name matches:", len(exact_matches))
print("Transfermarkt names not found in FBref:",
      len(transfer_names - fbref_names_all))

unmatched_all = sorted(transfer_names - fbref_names_all)

print("\nSample unmatched names:")
print(unmatched_all[:30])

```

Transfermarkt players: 138
 All FBref players: 2702
 Exact name matches: 87
 Transfermarkt names not found in FBref: 51

Sample unmatched names:
 ['Angus Gunn', 'Antoni Milambo', 'Antoñito Cordero', 'Arthur Masuaku', 'Bashir H

9.1 Verified Name Corrections

The following corrections were identified during the original matching process. These are explicit mappings rather than automatic fuzzy substitutions.

```

name_mapping = {
    "Dário Essugo": "Dario Essugo",
    "Hugo Ekitiké": "Hugo Ekitike",
    "James McAtee": "James Mcatee",
    "Yérémy Pino": "Yeremi Pino",
}

transfer_2025_clean["fbref_name"] = (
    transfer_2025_clean["player_name"]
    .replace(name_mapping)
)

matched_names = set(
    transfer_2025_clean.loc[
        transfer_2025_clean["fbref_name"].isin(fbref_names_all),
        "fbref_name"
    ]
)

print("Transfermarkt players:", transfer_2025_clean["player_name"].nunique())
print("Matched players after verified name corrections:", len(matched_names))

```

Transfermarkt players: 138
 Matched players after verified name corrections: 91

10. Select the Matched FBref Records

Some players appear more than once in FBref because they played for multiple clubs or competitions during the season.

For example, a player may have one row for a Premier League club and another row for a club in another league.

The matched rows are therefore aggregated at player level before joining them to the transfer data.


```
fbref_matched = df_fbref[
    df_fbref["Player"].isin(matched_names)
].copy()

print("Matched FBref rows:", len(fbref_matched))
print("Unique matched players:", fbref_matched["Player"].nunique())

duplicates = (
    fbref_matched["Player"]
    .value_counts()
    .loc[lambda x: x > 1]
)

print("\nPlayers with multiple FBref rows:")
display(duplicates)
```

Matched FBref rows: 98
Unique matched players: 91

Players with multiple FBref rows:

	count
Player	
Carlos Alcaraz	2
Adam Aznou	2
Jacob Bruun Larsen	2
Kevin Danso	2
Mathys Tel	2
Noah Okafor	2
Michael Kayode	2

dtype: int64

11. Select Performance Features

The model uses a mixture of:

- playing time
- attacking production
- expected-goal metrics
- shooting
- passing
- progression
- defensive actions
- possession / receiving actions

The raw transfer fee remains the target variable.

```
performance_features = [
    "MP", "Starts", "Min", "90s",
    "Gls", "Ast", "xG", "xAG", "npxG",
    "Sh", "SoT",
    "Cmp", "Att", "KP", "PPA", "xA",
    "PrgP", "PrgC", "PrgR",
    "Tkl", "TklW", "Int", "Tkl+Int", "Clr",
    "Blocks_stats_defense",
    "Touches", "Carries", "Succ", "Rec"
```

```
]

identity_features = ["Player", "Nation", "Pos", "Age"]

fbref_selected = fbref_matched[
    identity_features + performance_features
].copy()

fbref_aggregated = (
    fbref_selected
    .groupby("Player", as_index=False)
    .agg({
        "Nation": "first",
        "Pos": "first",
        "Age": "first",
        **{col: "sum" for col in performance_features}
    })
)

print("Rows after aggregation:", len(fbref_aggregated))
print("Unique players:", fbref_aggregated["Player"].nunique())

display(fbref_aggregated.head())
```

Rows after aggregation: 91
Unique players: 91

	Player	Nation	Pos	Age	MP	Starts	Min	90s	Gls	Ast	xG	xAG	np
0	Adam Aznou	ma MAR	DF	18.0	15	10	893	9.9	0	0	0.3	0.3	0
1	Adrien Truffert	fr FRA	DF,MF	22.0	33	31	2752	30.6	2	2	2.4	3.2	2
2	Alejandro Garnacho	ar ARG	MF,FW	20.0	36	23	2199	24.4	6	2	7.3	4.5	7
3	Alexander Isak	se SWE	FW	24.0	34	34	2756	30.6	23	6	20.3	4.3	17
4	Amine Adli	ma MAR	MF,FW	24.0	20	6	766	8.5	2	0	2.7	0.7	2

✓ 12. Merge Transfer Fees with Player Performance

The transfer table provides the target (**fee**) and transfer context.

FBref provides the performance features.

An inner join is used so that only players with both a valid transfer record and a matched performance record enter the modeling dataset.

```
model_data = transfer_2025_clean.merge(
    fbref_aggregated,
    left_on="fbref_name",
    right_on="Player",
    how="inner"
)

print("Model dataset shape:", model_data.shape)
print("Unique players:", model_data["Player"].nunique())
print("Missing transfer fees:", model_data["fee"].isna().sum())

display(model_data.head())
```

Model dataset shape: (91, 50)
Unique players: 91
Missing transfer fees: 0

	season	league	club	window	movement	player_name	player_id	age	r
0	2025	Premier League	AFC Bournemouth	summer	in	Bafodé Diakité	463605	24	
1	2025	Premier League	AFC Bournemouth	summer	in	Amine Adli	532776	25	
2	2025	Premier League	AFC Bournemouth	summer	in	Adrien Truffert	654539	23	
3	2025	Premier League	Arsenal FC	summer	in	Martín Zubimendi	423440	26	
4	2025	Premier League	Arsenal FC	summer	in	Eberechi Eze	479999	27	

13. Transfer Fee Distribution

The final matched dataset contains **91 players**.

The transfer-fee target is strongly right-skewed: a small number of expensive transfers are much larger than the majority of observations.

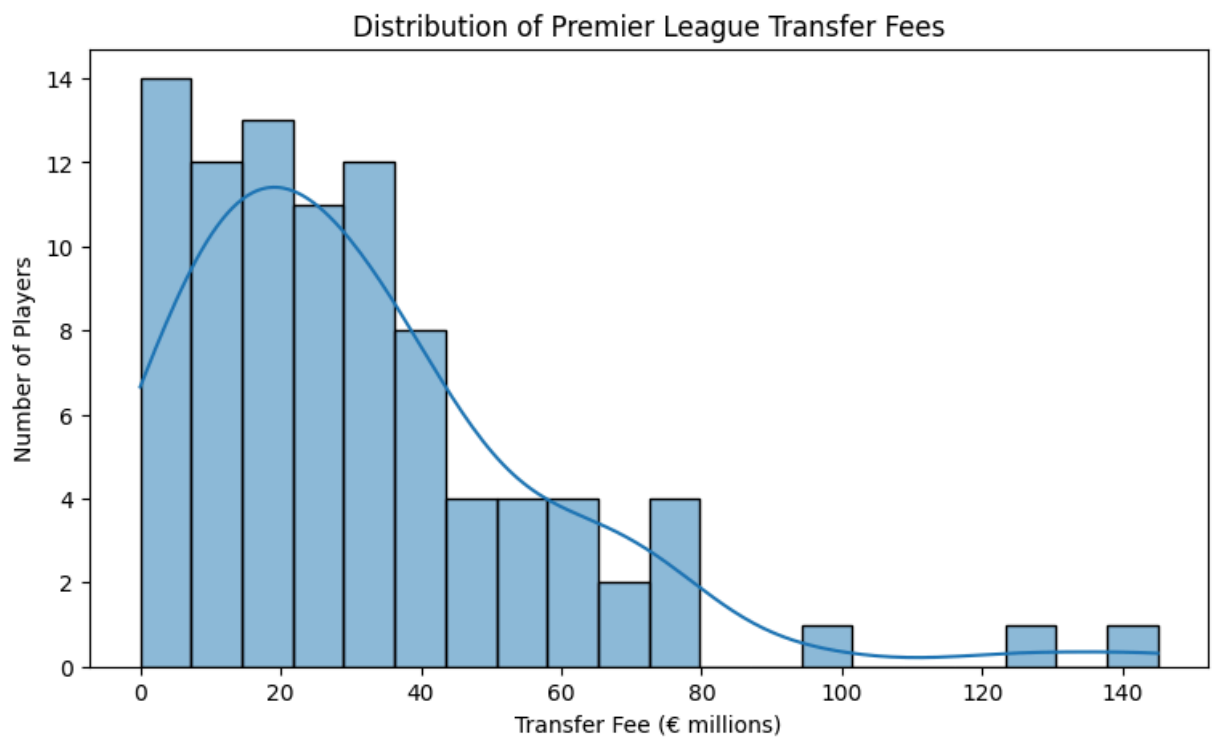
This matters because metrics such as RMSE are sensitive to large errors on expensive transfers.

```
print("Transfer fee statistics:")
display(model_data["fee"].describe())

print("Zero-fee transfers:", (model_data["fee"] == 0).sum())
print("Non-zero transfers:", (model_data["fee"] > 0).sum())
```

Transfer fee statistics:	
	fee
count	9.100000e+01
mean	3.121121e+07
std	2.680143e+07
min	0.000000e+00
25%	1.325000e+07
50%	2.500000e+07
75%	4.240000e+07
max	1.450000e+08
dtype: float64	
Zero-fee transfers: 7	
Non-zero transfers: 84	

```
plt.figure(figsize=(9, 5))
sns.histplot(model_data["fee"] / 1_000_000, bins=20, kde=True)
plt.xlabel("Transfer Fee (€ millions)")
plt.ylabel("Number of Players")
plt.title("Distribution of Premier League Transfer Fees")
plt.show()
```



✓ 14. Highest-Value Transfers

Looking at the largest transfer fees provides a useful sanity check on the target data.

```
top_transfers = (  
    model_data[  
        ["player_name", "club", "dealing_club", "fee", "market_value"]  
    ]  
    .sort_values("fee", ascending=False)  
    .head(20)  
    .copy()  
)  
  
top_transfers["fee_m"] = top_transfers["fee"] / 1_000_000  
top_transfers["market_value_m"] = top_transfers["market_value"] / 1_000_000  
  
display(  
    top_transfers[  
        ["player_name", "club", "dealing_club", "fee_m", "market_value_m"]  
    ]  
)
```

	player_name	club	dealing_club	fee_m	market_value_m	
47	Alexander Isak	Liverpool FC	Newcastle United	145.00	120.0	
48	Florian Wirtz	Liverpool FC	Bayer 04 Leverkusen	125.00	140.0	
49	Hugo Ekitiké	Liverpool FC	Eintracht Frankfurt	95.00	75.0	
57	Benjamin Šeško	Manchester United	RB Leipzig	76.50	70.0	
58	Bryan Mbeumo	Manchester United	Brentford FC	75.00	55.0	
60	Nick Woltemade	Newcastle United	VfB Stuttgart	75.00	30.0	
59	Matheus Cunha	Manchester United	Wolverhampton Wanderers	74.20	60.0	
3	Martín Zubimendi	Arsenal FC	Real Sociedad	70.00	60.0	
4	Eberechi Eze	Arsenal FC	Crystal Palace	69.30	55.0	
78	Xavi Simons	Tottenham Hotspur	RB Leipzig	65.00	70.0	
79	Mohammed Kudus	Tottenham Hotspur	West Ham United	63.80	45.0	
21	João Pedro	Chelsea FC	Brighton & Hove Albion	63.70	50.0	
61	Anthony Elanga	Newcastle United	Nottingham Forest	61.40	42.0	
62	Yoane Wissa	Newcastle United	Brentford FC	57.70	32.0	

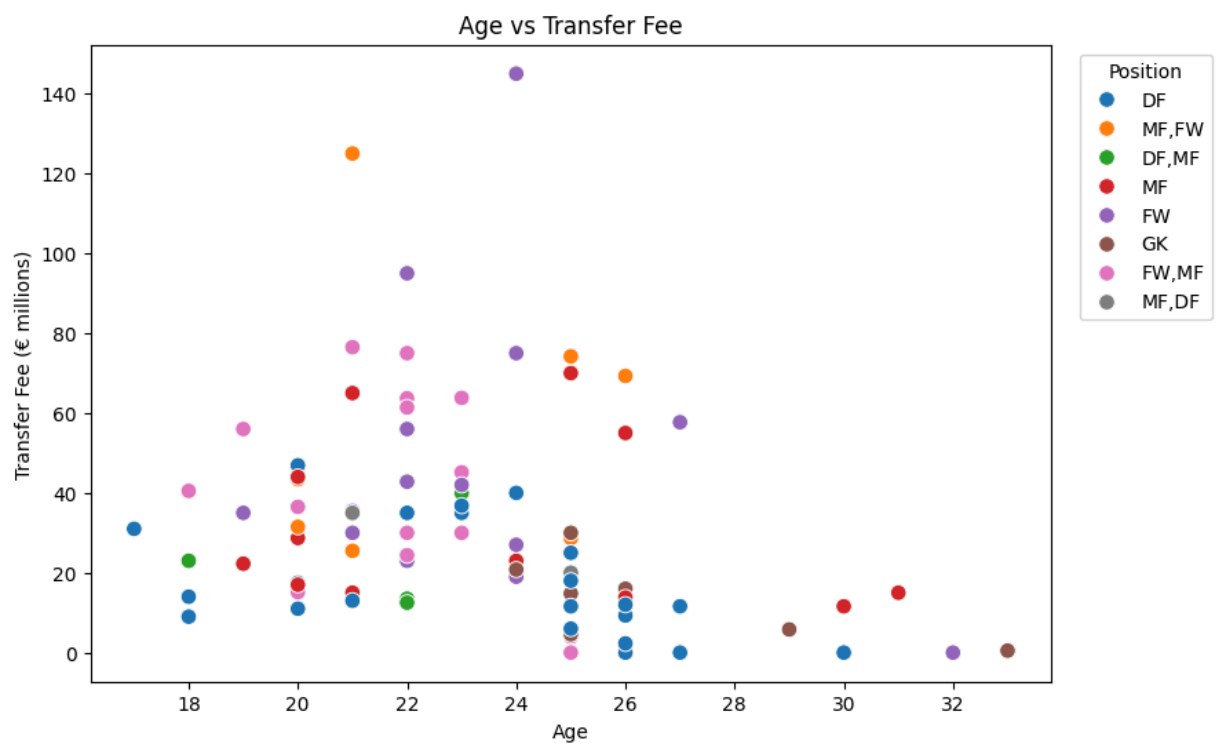
✓ 15. Exploratory Analysis

Age vs Transfer Fee

Age is included because transfer valuation can vary substantially across age groups.

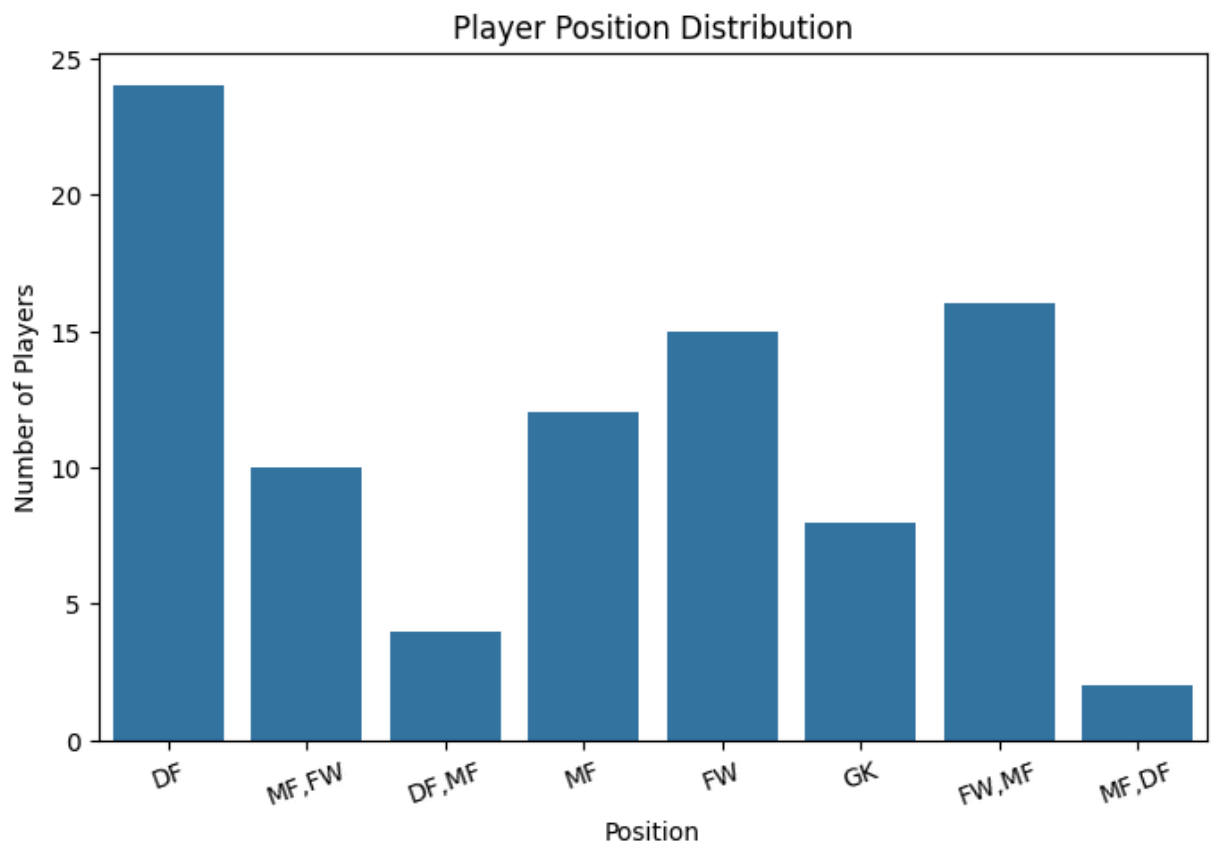
This plot is descriptive rather than causal: it does not establish that age alone determines a player's fee.

```
plt.figure(figsize=(9, 6))
sns.scatterplot(
    data=model_data,
    x="Age",
    y=model_data["fee"] / 1_000_000,
    hue="Pos",
    s=70
)
plt.xlabel("Age")
plt.ylabel("Transfer Fee (€ millions)")
plt.title("Age vs Transfer Fee")
plt.legend(title="Position", bbox_to_anchor=(1.02, 1), loc="upper left")
plt.show()
```



Position Distribution

```
plt.figure(figsize=(8, 5))
sns.countplot(data=model_data, x="Pos")
plt.xlabel("Position")
plt.ylabel("Number of Players")
plt.title("Player Position Distribution")
plt.xticks(rotation=20)
plt.show()
```



Selected Performance Relationships

The following plots inspect several variables that may contain useful information about transfer fees.

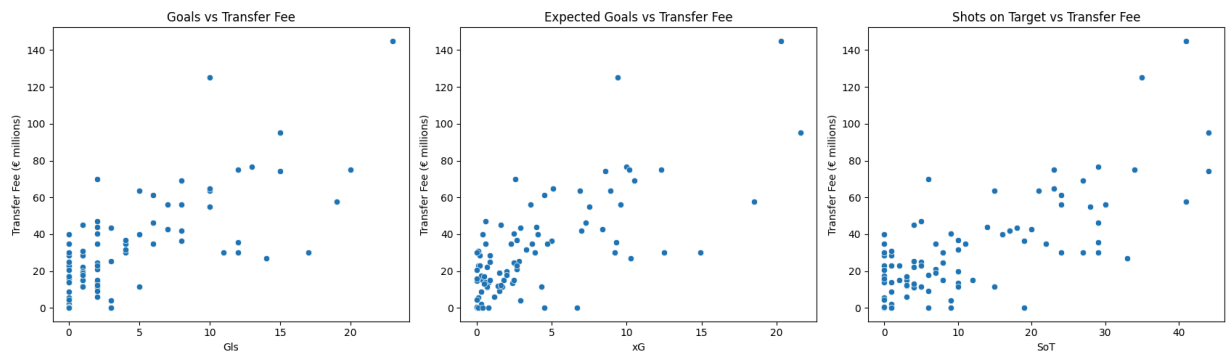
```
fig, axes = plt.subplots(1, 3, figsize=(17, 5))

sns.scatterplot(data=model_data, x="Gls", y=model_data["fee"] / 1_000_000, ax=axes[0])
axes[0].set_title("Goals vs Transfer Fee")
axes[0].set_ylabel("Transfer Fee (€ millions)")

sns.scatterplot(data=model_data, x="xG", y=model_data["fee"] / 1_000_000, ax=axes[1])
axes[1].set_title("Expected Goals vs Transfer Fee")
axes[1].set_ylabel("Transfer Fee (€ millions)")

sns.scatterplot(data=model_data, x="SoT", y=model_data["fee"] / 1_000_000, ax=axes[2])
axes[2].set_title("Shots on Target vs Transfer Fee")
axes[2].set_ylabel("Transfer Fee (€ millions)")

plt.tight_layout()
plt.show()
```



✓ 16. Build the Final Modeling Table

The modeling table keeps only:

- player name
- transfer fee
- selected FBref identity/performance variables

`player_name` is retained for reporting but is **not used as a predictive feature**.

```
id_columns = ["player_name", "fee"]

fbref_model_features = [
    "Age", "Pos",
    "MP", "Starts", "Min", "90s",
    "Gls", "Ast", "xG", "xAG", "npxG",
    "Sh", "SoT",
    "Cmp", "Att", "KP", "PPA", "xA",
    "PrgP", "PrgC", "PrgR",
    "Tkl", "TklW", "Int", "Tkl+Int", "Clr",
    "Blocks_stats_defense",
    "Touches", "Carries", "Succ", "Rec"
]

model_df = model_data[
    id_columns + fbref_model_features
].copy()

print("Final dataset shape:", model_df.shape)
display(model_df.head())
```


Final dataset shape: (91, 33)

	player_name	fee	Age	Pos	MP	Starts	Min	90s	Gls	Ast	xG	xG
0	Bafodé Diakité	35000000.0	23.0	DF	31	31	2766	30.7	4	1	2.3	0
1	Amine Adli	21000000.0	24.0	MF,FW	20	6	766	8.5	2	0	2.7	0
2	Adrien Truffert	13500000.0	22.0	DF,MF	33	31	2752	30.6	2	2	2.4	3
3	Martín Zubimendi	70000000.0	25.0	MF	36	33	2962	32.9	2	1	2.6	1
4	Eberechi Eze	69300000.0	26.0	MF,FW	34	31	2588	28.8	8	8	10.5	6

17. Missing Values

Only one missing value was present in **Age** in the original workflow.

Median imputation is used so that the model receives a complete numeric feature matrix without dropping an otherwise usable player.

```
print("Missing values before imputation:")
display(model_df.isna().sum()[lambda s: s > 0])

model_df["Age"] = model_df["Age"].fillna(model_df["Age"].median())

print("Total missing values after imputation:",
      model_df.isna().sum().sum())
print("Median age used:", model_df["Age"].median())
```

Missing values before imputation:

	0
--	---

Age	1
-----	---

dtype: int64
Total missing values after imputation: 0
Median age used: 23.0

18. Define Features and Target

```
X = model_df.drop(columns=["player_name", "fee"])
y = model_df["fee"]

print("X shape:", X.shape)
print("y shape:", y.shape)

print("\nTarget statistics:")
display(y.describe())
```

```
X shape: (91, 31)
y shape: (91,)

Target statistics:

```

	fee
count	9.100000e+01
mean	3.121121e+07
std	2.680143e+07
min	0.000000e+00
25%	1.325000e+07
50%	2.500000e+07
75%	4.240000e+07
max	1.450000e+08

```
dtype: float64
```

✓ 19. Establish a Baseline

Before training a machine-learning model, we compare against a simple baseline that predicts the mean transfer fee for every player.

A useful model should provide meaningful improvement over this baseline.

```
kf = KFold(
    n_splits=5,
    shuffle=True,
    random_state=RANDOM_STATE
)

baseline_model = DummyRegressor(strategy="mean")

baseline_results = cross_validate(
    baseline_model,
    X,
    y,
    cv=kf,
    scoring={
        "MAE": "neg_mean_absolute_error",
        "RMSE": "neg_root_mean_squared_error",
        "R2": "r2"
    },
    n_jobs=-1
)

baseline_mae = -baseline_results["test_MAE"]
baseline_rmse = -baseline_results["test_RMSE"]
baseline_r2 = baseline_results["test_R2"]

print("5-Fold Baseline")
print("-----")
print(f"MAE : €{baseline_mae.mean():.2f} ± €{baseline_mae.std():.2f}")
print(f"RMSE: €{baseline_rmse.mean():.2f} ± €{baseline_rmse.std():.2f}")
print(f"R² : {baseline_r2.mean():.4f} ± {baseline_r2.std():.4f}")
```

```
5-Fold Baseline
-----
MAE : €20,055,753.93 ± €885,934.77
```

RMSE: €26,589,056.52 ± €4,089,303.94
R² : -0.0657 ± 0.0901

✓ 20. Prepare the Random Forest Pipeline

`Pos` is categorical, so it is one-hot encoded.

The numerical variables are passed through unchanged.

Keeping preprocessing inside a scikit-learn pipeline prevents the transformation logic from becoming disconnected from model training.

```
categorical_features = ["Pos"]

numeric_features = [
    col for col in X.columns
    if col not in categorical_features
]

preprocessor = ColumnTransformer(
    transformers=[
        (
            "categorical",
            OneHotEncoder(handle_unknown="ignore"),
            categorical_features
        ),
        (
            "numeric",
            "passthrough",
            numeric_features
        )
    ]
)

rf_model = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        (
            "model",
            RandomForestRegressor(
                n_estimators=300,
                random_state=RANDOM_STATE,
                n_jobs=-1
            )
        )
    ]
)

print("Random Forest pipeline created successfully.")
```

Random Forest pipeline created successfully.

✓ 21. Initial Random Forest

A Random Forest is appropriate for this project because it can model nonlinear relationships and interactions between player characteristics without requiring a linear relationship between each input and transfer fee.

```
initial_cv = cross_validate(
    rf_model,
    X,
```

```

y,
cv=kf,
scoring={
    "MAE": "neg_mean_absolute_error",
    "RMSE": "neg_root_mean_squared_error",
    "R2": "r2"
},
n_jobs=-1
)

initial_mae = -initial_cv["test_MAE"]
initial_rmse = -initial_cv["test_RMSE"]
initial_r2 = initial_cv["test_R2"]

print("Initial Random Forest - 5-Fold CV")
print("-----")
print(f"MAE : €{initial_mae.mean():.2f} ± €{initial_mae.std():.2f}")
print(f"RMSE: €{initial_rmse.mean():.2f} ± €{initial_rmse.std():.2f}")
print(f"R² : {initial_r2.mean():.4f} ± {initial_r2.std():.4f}")

```

```

Initial Random Forest - 5-Fold CV
-----
MAE : €13,577,285.95 ± €2,375,274.50
RMSE: €18,540,559.76 ± €4,011,051.60
R² : 0.4670 ± 0.1819

```

✓ 22. Log-Target Experiment

Transfer fees are heavily right-skewed, so we also test a log-transformed target.

`log1p` safely handles the seven zero-fee observations because `log1p(0) = 0`.

The model is evaluated on the original euro scale after reversing the transformation.

```

from sklearn.compose import TransformedTargetRegressor

rf_log_base = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        (
            "model",
            RandomForestRegressor(
                n_estimators=300,
                random_state=RANDOM_STATE,
                n_jobs=-1
            )
        )
    ]
)

rf_log_model = TransformedTargetRegressor(
    regressor=rf_log_base,
    func=np.log1p,
    inverse_func=np.expm1
)

log_cv = cross_validate(
    rf_log_model,
    X,
    y,
    cv=kf,
    scoring={
        "MAE": "neg_mean_absolute_error",
        "RMSE": "neg_root_mean_squared_error",
        "R2": "r2"
    }
)

```

```

    },
    n_jobs=-1
)

print("Random Forest + Log Target - 5-Fold CV")
print("-----")
print(f"MAE : €{-log_cv['test_MAE'].mean():.2f}")
print(f"RMSE: €{-log_cv['test_RMSE'].mean():.2f}")
print(f"R² : {-log_cv['test_R2'].mean():.4f}")

```

```

Random Forest + Log Target - 5-Fold CV
-----
MAE : €14,072,733.25
RMSE: €20,622,875.77
R² : 0.3373

```

✓ 23. Reduce Redundant Features

The first model uses many related volume statistics. To make the final model more compact and easier to explain, a reduced feature set is tested.

The reduced set focuses on:

- age and position
- playing time
- goals and assists
- expected goals / assists
- shooting
- key passes
- progression
- defensive actions

```

reduced_features = [
    "Age", "Pos", "Min",
    "Gls", "Ast", "xG", "xAG",
    "Sh", "SoT", "KP",
    "PrgP", "PrgC", "PrgR",
    "Succ", "TklW", "Tkl", "Int", "Clr"
]

X_reduced = model_df[reduced_features]
y = model_df["fee"]

categorical_features_reduced = ["Pos"]
numeric_features_reduced = [
    col for col in X_reduced.columns
    if col not in categorical_features_reduced
]

preprocessor_reduced = ColumnTransformer(
    transformers=[
        (
            "categorical",
            OneHotEncoder(handle_unknown="ignore"),
            categorical_features_reduced
        ),
        (
            "numeric",
            "passthrough",
            numeric_features_reduced
        )
    ]
)

```

```

)

rf_reduced = Pipeline(
    steps=[
        ("preprocessor", preprocessor_reduced),
        (
            "model",
            RandomForestRegressor(
                n_estimators=300,
                random_state=RANDOM_STATE,
                n_jobs=-1
            )
        )
    ]
)

reduced_cv = cross_validate(
    rf_reduced,
    X_reduced,
    y,
    cv=kf,
    scoring={
        "MAE": "neg_mean_absolute_error",
        "RMSE": "neg_root_mean_squared_error",
        "R2": "r2"
    },
    n_jobs=-1
)

print("Reduced Random Forest - 5-Fold CV")
print("-----")
print(f"MAE : €{-reduced_cv['test_MAE'].mean():.2f}")
print(f"RMSE: €{-reduced_cv['test_RMSE'].mean():.2f}")
print(f"R² : {reduced_cv['test_R2'].mean():.4f}")

```

```

Reduced Random Forest - 5-Fold CV
-----
MAE : €13,360,330.57
RMSE: €18,516,935.23
R² : 0.4692

```

✓ 24. Tune the Reduced Random Forest

Hyperparameters are selected using cross-validation.

Parameters tested

- `max_depth`
- `min_samples_split`
- `min_samples_leaf`
- `max_features`

The optimization objective is **MAE**, because it is directly interpretable as the average euro error.

```

param_grid = {
    "model__max_depth": [None, 5, 10, 15],
    "model__min_samples_split": [2, 5, 10],
    "model__min_samples_leaf": [1, 2, 4],
    "model__max_features": [0.7, 1.0]
}

grid_search = GridSearchCV(

```

```

        estimator=rf_reduced,
        param_grid=param_grid,
        scoring="neg_mean_absolute_error",
        cv=kf,
        n_jobs=-1,
        verbose=1
    )

    grid_search.fit(X_reduced, y)

    print("Best parameters:")
    print(grid_search.best_params_)
    print(f"\nBest cross-validation MAE: €{-grid_search.best_score_:.2f}")

```

Fitting 5 folds for each of 72 candidates, totalling 360 fits
 Best parameters:
 {'model__max_depth': 5, 'model__max_features': 0.7, 'model__min_samples_leaf': 1
 Best cross-validation MAE: €12,973,289.29

✓ 25. Tuned Model Cross-Validation

The tuned model is evaluated again with the same 5 folds to obtain MAE, RMSE, and R^2 together.

```

best_rf = grid_search.best_estimator_

tuned_cv = cross_validate(
    best_rf,
    X_reduced,
    y,
    cv=kf,
    scoring={
        "MAE": "neg_mean_absolute_error",
        "RMSE": "neg_root_mean_squared_error",
        "R2": "r2"
    },
    n_jobs=-1
)

tuned_mae = -tuned_cv["test_MAE"]
tuned_rmse = -tuned_cv["test_RMSE"]
tuned_r2 = tuned_cv["test_R2"]

print("Tuned Random Forest - 5-Fold CV")
print("-----")
print(f"MAE : €{tuned_mae.mean():.2f} ± €{tuned_mae.std():.2f}")
print(f"RMSE: €{tuned_rmse.mean():.2f} ± €{tuned_rmse.std():.2f}")
print(f"R² : {tuned_r2.mean():.4f} ± {tuned_r2.std():.4f}")
print("\nFold R² scores:", np.round(tuned_r2, 4))

```

Tuned Random Forest - 5-Fold CV

 MAE : €12,973,289.29 ± €2,795,792.46
 RMSE: €17,833,555.21 ± €4,306,002.10
 R² : 0.5085 ± 0.1666
 Fold R² scores: [0.7682 0.4581 0.2531 0.4966 0.5665]

✓ 26. Final Holdout Evaluation

To avoid reporting the same observations used during model development as an independent test, the final model is evaluated on a held-out 20% test set.

Note: Because this cleaned notebook performs hyperparameter selection on the full dataset through cross-validation before the holdout split, the holdout score below should be treated as an illustrative project check rather than a fully nested, publication-grade estimate. A nested CV setup would be preferable for a rigorous research evaluation.

```
X_train, X_test, y_train, y_test = train_test_split(
    X_reduced,
    y,
    test_size=0.20,
    random_state=RANDOM_STATE
)

best_rf.fit(X_train, y_train)
y_pred = best_rf.predict(X_test)

final_mae = mean_absolute_error(y_test, y_pred)
final_rmse = mean_squared_error(y_test, y_pred) ** 0.5
final_r2 = r2_score(y_test, y_pred)

print("Holdout Test Results")
print("-----")
print(f"MAE : €{final_mae:,.2f}")
print(f"RMSE: €{final_rmse:,.2f}")
print(f"R² : {final_r2:.4f}")
```

Holdout Test Results

MAE : €9,126,053.17
RMSE: €11,374,137.33
R² : 0.7764

✓ 27. Actual vs Predicted Transfer Fees

A model that predicts well should place observations relatively close to the diagonal `Actual = Predicted` line.

```
results_df = pd.DataFrame({
    "Player": model_df.loc[X_test.index, "player_name"].values,
    "Actual Fee": y_test.values,
    "Predicted Fee": y_pred
})

results_df["Absolute Error"] = (
    results_df["Actual Fee"] - results_df["Predicted Fee"]
).abs()

results_df["Error %"] = (
    results_df["Absolute Error"] /
    results_df["Actual Fee"].replace(0, np.nan)
) * 100

display(
    results_df.sort_values("Actual Fee", ascending=False)
)

plt.figure(figsize=(8, 6))
plt.scatter(y_test / 1_000_000, y_pred / 1_000_000, s=70)
```

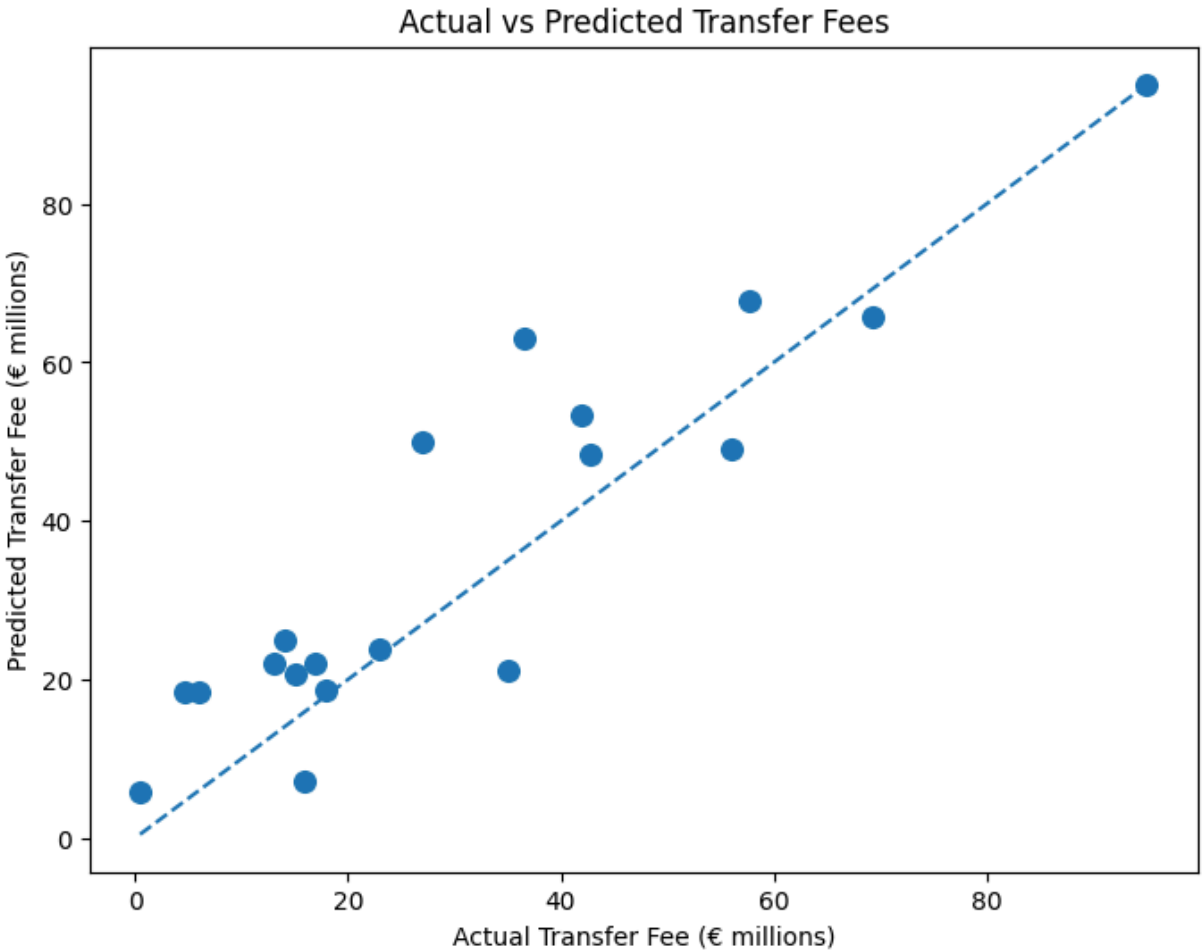


```
min_value = min(y_test.min(), y_pred.min()) / 1_000_000
max_value = max(y_test.max(), y_pred.max()) / 1_000_000

plt.plot(
    [min_value, max_value],
    [min_value, max_value],
    linestyle="--"
)

plt.xlabel("Actual Transfer Fee (€ millions)")
plt.ylabel("Predicted Transfer Fee (€ millions)")
plt.title("Actual vs Predicted Transfer Fees")
plt.show()
```

	Player	Actual Fee	Predicted Fee	Absolute Error	Error %
18	Hugo Ekitiké	95000000.0	9.504809e+07	4.808746e+04	0.050618
15	Eberechi Eze	69300000.0	6.574630e+07	3.553702e+06	5.127997
13	Yoane Wissa	57700000.0	6.784341e+07	1.014341e+07	17.579569
1	Jamie Gittens	56000000.0	4.912136e+07	6.878640e+06	12.283287
14	Dango Ouattara	42800000.0	4.832034e+07	5.520345e+06	12.898002
7	Dan Ndoye	42000000.0	5.339867e+07	1.139867e+07	27.139681
2	Rayan Cherki	36500000.0	6.311646e+07	2.661646e+07	72.921800
4	Bafodé Diakité	35000000.0	2.111761e+07	1.388239e+07	39.663975
3	Jørgen Strand Larsen	27000000.0	4.985499e+07	2.285499e+07	84.648130
17	Jaydee Canvot	23000000.0	2.390536e+07	9.053626e+05	3.936359
6	Jaka Bijol	18000000.0	1.861108e+07	6.110781e+05	3.394878
10	Soungoutou Magassa	17000000.0	2.200658e+07	5.006581e+06	29.450477
0	Lucas Perri	16000000.0	7.283155e+06	8.716845e+06	54.480280
16	Loum Tchaoua	15150000.0	2.065810e+07	5.508104e+06	36.357121
5	Mamadou Sarr	14000000.0	2.490290e+07	1.090290e+07	77.877863
12	Nicolò Savona	13000000.0	2.206262e+07	9.062622e+06	69.712478
9	Sebastiaan Bornauw	6000000.0	1.846742e+07	1.246742e+07	207.790391
11	Mark Travers	4600000.0	1.848355e+07	1.388355e+07	301.816327
8	Marco Bizot	500000.0	5.933847e+06	5.433847e+06	1086.769402



28. Permutation Feature Importance

Permutation importance measures how much model performance changes when one feature is randomly shuffled.

This is useful for understanding which variables the fitted model relies on most. It should **not** be interpreted as proof that a feature causes transfer prices to change.

```
best_rf.fit(X_reduced, y)

final_importance = permutation_importance(
    best_rf,
    X_reduced,
    y,
    scoring="r2",
    n_repeats=20,
    random_state=RANDOM_STATE,
    n_jobs=-1
)

final_importance_df = pd.DataFrame({
    "Feature": X_reduced.columns,
    "Importance": final_importance.importances_mean,
    "Std": final_importance.importances_std
}).sort_values("Importance", ascending=False)

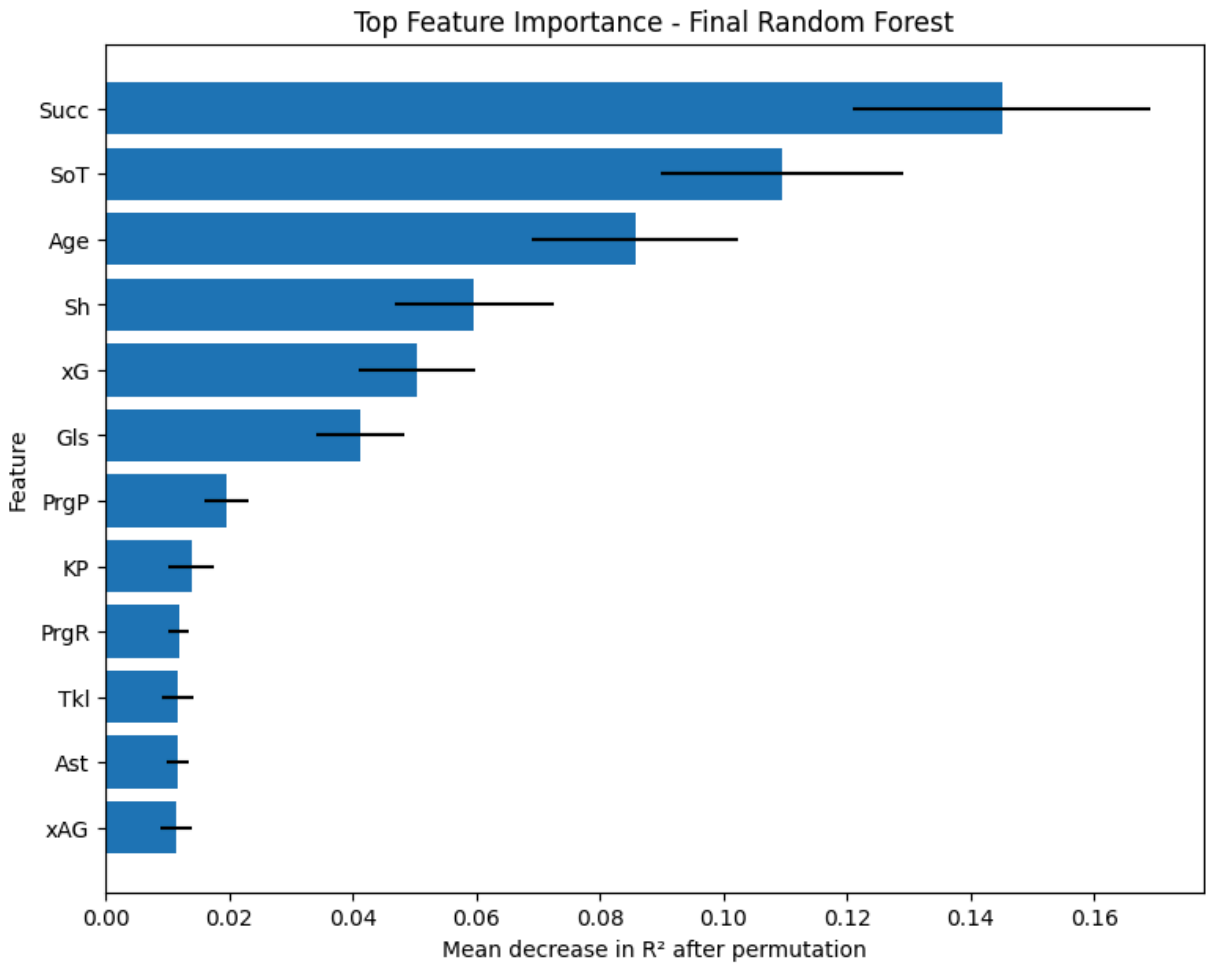
display(final_importance_df)

plt.figure(figsize=(9, 7))
plot_df = final_importance_df.head(12).sort_values("Importance")

plt.barh(
    plot_df["Feature"],
    plot_df["Importance"],
    xerr=plot_df["Std"]
)

plt.xlabel("Mean decrease in R2 after permutation")
plt.ylabel("Feature")
plt.title("Top Feature Importance - Final Random Forest")
plt.show()
```

	Feature	Importance	Std	
13	Succ	0.145084	0.024121	
8	SoT	0.109515	0.019504	
0	Age	0.085737	0.016720	
7	Sh	0.059624	0.012839	
5	xG	0.050417	0.009401	
3	Gls	0.041168	0.007170	
10	PrgP	0.019714	0.003583	
9	KP	0.013883	0.003607	
12	PrgR	0.011866	0.001703	
15	Tkl	0.011675	0.002498	
4	Ast	0.011609	0.001794	
6	xAG	0.011452	0.002556	
14	TklW	0.011435	0.002587	
17	Clr	0.010080	0.002890	
2	Min	0.008360	0.001134	
11	PrgC	0.006813	0.001874	
16	Int	0.004929	0.000964	
1	Pos	0.004409	0.000775	



Next steps:

[Generate code with final_importance_df](#)

[New interactive sheet](#)

Observed Feature-Importance Pattern

In the original project run, the strongest permutation-importance variables included:

- **Succ** — successful take-ons
- **SoT** — shots on target
- **Age**
- **Sh** — shots
- **xG** — expected goals
- **Gls** — goals

The exact values can change slightly when the notebook is rerun because the training/evaluation sample is small.

These results should be interpreted as **model associations**, not as evidence that any single statistic determines a player's transfer fee.

✓ 29. Test the Model on an Individual Player

The function below accepts the reduced set of model inputs and returns an estimated transfer fee.

For consistency, it uses exactly the same feature names and preprocessing pipeline as the trained model.

```
def predict_transfer_value(
    age,
    position,
    minutes,
    goals,
    assists,
    xg,
    xag,
    shots,
    shots_on_target,
    key_passes,
    progressive_passes,
    progressive_carries,
    progressive_receives,
    successful_takeons,
    tackles_won,
    tackles,
    interceptions,
    clearances
):
    player_input = pd.DataFrame([{
        "Age": age,
        "Pos": position,
        "Min": minutes,
        "Gls": goals,
        "Ast": assists,
        "xG": xg,
        "xAG": xag,
        "Sh": shots,
        "SoT": shots_on_target,
        "KP": key_passes,
        "PrgP": progressive_passes,
```

```
        "PrgC": progressive_carries,
        "PrgR": progressive_receives,
        "Succ": successful_takeons,
        "TklW": tackles_won,
        "Tkl": tackles,
        "Int": interceptions,
        "Clr": clearances
    })

prediction = best_rf.predict(player_input)[0]

print("=====")
print(" PREMIER LEAGUE TRANSFER VALUE PREDICTOR")
print("=====")
print(f"Position: {position}")
print(f"Age: {age:.0f}")
print(f"Estimated Transfer Fee: €{prediction:,.0f}")
print(f"Estimated Transfer Fee: €{prediction / 1_000_000:.2f} million")
print("=====")
```